PHÂN TÍCH CHUYÊN SÂU · ELECTRONICS 2025, 14, 3993

Triển khai tối ưu YOLOv3-Tiny cho nhận dạng ảnh và video thời gian thực trên FPGA

Từ huấn luyện lượng tử hóa đến bitstream, tối ưu folding, công suất và so sánh

Riccardo Calì, Laura Falaschetti, Giorgio Biagetti

Università Politecnica delle Marche, Ancona, Ý · doi:10.3390/electronics14203993

NỘI DUNG

Lộ trình bài trình bày

Bối cảnh và công cụ

Slide 3–10: động lực, nền tảng suy luận, khoảng trống, so sánh toolflow, FINN

Phương pháp

Slide 11–26: nền tảng, mô hình, QAT, biến đổi FINN, RTL/HLS, folding, FIFO, driver

Kết quả

Slide 27–38: độ chính xác, MAC, tài nguyên, hiệu năng, công suất, so sánh

Phân tích và kết luận

Slide 39–42: số liệu tự tính, phản biện, kết luận, câu hỏi dự kiến

TÓM TẮT

Một slide cho cả bài báo

208
FPS ở 200 MHz, batch 100

2,55 W
đo cả board, bench 5 V

81,9
FPS/W (cả board)

~150 €
giá board Z-Turn

Làm gì

- Huấn luyện QAT INT4/INT8
- FINN sinh IP, tối ưu folding
- Driver kiểu TPU qua Ethernet

So với ai

- Coral Edge TPU: tốc độ tương đương
- FPGA khác: FPS/W cao hơn
- LPYOLO cùng mạng: ~10× FPS

Đánh đổi

- mAP@0.5 chỉ 17,7% (gốc 41,8%)
- Driver Python mất 21–50% FPS
- Dùng 98,6% BRAM, 92,7% DSP

BỐI CẢNH

Ràng buộc của hệ nhúng và drone

Ràng buộc

- Công suất, tản nhiệt, chi phí
- Pin nhỏ, khối lượng tải (payload) hạn chế
- Hoạt động ngoài môi trường được kiểm soát

Lợi ích suy luận tại chỗ

- Không phụ thuộc đường truyền băng thông cao
- Độ trễ thấp, quyết định nhanh hơn
- Tự chủ khi mất kết nối

Ứng dụng nêu trong bài

- Nông nghiệp: bay trên diện tích lớn, phát hiện bệnh cây, chỉ ra vùng cần can thiệp
- Giám sát: nhận diện hành vi bất thường, cảnh báo ngay
- Cứu nạn: phát hiện người dưới đống đổ nát, rút ngắn thời gian phản ứng

GPU, NPU, TPU và FPGA

Nền tảng	Điểm mạnh	Hạn chế trong bối cảnh drone
GPU	Thông lượng cao, hỗ trợ framework rộng	Công suất, kích thước, tản nhiệt, giá
Jetson (GPU nhúng)	Hiệu năng/W tốt hơn GPU rời	Vẫn đắt và tốn điện cho thiết bị nhỏ
NPU / SoC tích hợp	INT8/INT4, bộ nhớ trên chip, hiệu năng/W tốt	Chỉ hỗ trợ tập toán tử hẹp
TPU / Coral Edge TPU	Mảng MAC cố định, tiết kiệm điện	Kiến trúc cố định; hỗ trợ chính thức giảm, do cộng đồng duy trì
FPGA	Tái cấu hình, song song, pipeline tùy biến, lượng tử hóa tùy ý	Chuỗi công cụ phức tạp, giới hạn tài nguyên chip

Hailo và các startup khác cũng đưa ra TPU công suất thấp ở mức vài TOPS.

NỀN TẢNG HỌC THUẬT

Mạng gọn và nén mô hình

Kiến trúc nhẹ

SqueezeNet, MobileNet, ShuffleNet/V2, EfficientNet. Với phát hiện đối tượng: YOLOv3 và biến thể YOLOv3-Tiny là mốc tham chiếu cho hệ nhúng.

Nén mô hình

Pruning, lượng tử hóa, chia sẻ trọng số. Giảm bộ nhớ và băng thông nhờ tính toán số nguyên thay vì dấu phẩy động.

Lượng tử hóa cực đoan

BNN, XNOR-Net, DoReFa-Net: dùng 1-2 bit, suy luận số nguyên cho phần cứng công suất thấp. Bài này chọn mức vừa phải: INT4 (INT8 ở lớp đầu và cuối).

Vì sao quan trọng với FPGA

Số bit nhỏ cho phép đóng gói nhiều MAC trong một DSP (4 MAC/DSP với 4 bit) và giữ toàn bộ trọng số trong BRAM trên chip.

Ba khoảng trống và cách bài báo trả lời

Khoảng trống (mục 2.5)	Đáp ứng trong bài
(i) Hầu hết FPGA accelerator nhắm thiết bị cao cấp	Zynq-7020: ~100–150 €, vài watt
(ii) Chuỗi công cụ huấn luyện đến triển khai còn thiếu	Brevitas, QONNX, FINN, PYNQ, driver .fnn, mã nguồn công khai
(iii) Ít báo cáo FPS/W có hệ thống	Đo công suất thật ở 3 tần số, so với Coral và 4 công trình FPGA

Phạm vi: không tối đa hóa độ chính xác. Mô hình chỉ là vật thử để đẩy tối đa tài nguyên và thông lượng, và để chứng minh quy trình.

CÔNG CỤ FPGA

So sánh các toolflow cho Zynq-7000

Công cụ	Mô hình	Vấn đề với Zynq-7020	Kết luận
Vitis AI (DPU)	Lõi MAC cố định	Ngừng hỗ trợ Zynq-7000; thiết bị đích trên 10 W, từ 300 $	Loại
NVDLA	Mã nguồn mở, IP dựng sẵn	Vài FPS cho AlexNet ngay cả Kintex-7	Loại
NN2FPGA	Streaming, chỉ HLS	Chưa có ví dụ trên board nhỏ; flow chưa chín	Loại
fpgaConvNet	Streaming + tái cấu hình từng phần	Tốn LUT/BRAM/DSP; reconfig vài chục ms; YOLOv3-Tiny bị hạn chế	Loại
FINN 0.10	**Streaming, RTL + HLS**	**Chín muồi, hệ sinh thái tích cực, RTL tiết kiệm logic**	**Chọn**

Streaming dataflow: mỗi lớp một mô-đun

QONNX
Mạng lượng tử hóa, kèm thông tin bit

Biến đổi
Mỗi nút thành mô-đun phần cứng

Mô-đun
HLS (Vitis) hoặc RTL (SystemVerilog)

IP ghép
Nối bằng luồng AXI-Stream

Bitstream
Vivado tổng hợp, place & route

Ưu điểm cho dự án

- Toàn bộ pipeline nằm trên chip, ít truy cập DRAM
- Hỗ trợ lượng tử hóa cực thấp
- Có RTL tối ưu (DSP packing, parallel window)

Hạn chế gặp phải

- Không xử lý mạng rẽ nhánh
- Chỉ một dataflow partition trên PL
- HLS StreamingMaxPool kém hiệu quả trên bản đồ lớn

MYiR Z-Turn Board V2: XC7Z020CLG400-2

Hệ xử lý (PS)	Giá trị
CPU	2× ARM Cortex-A9, 750 MHz
L1 / L2	2×32 KiB (D/I) mỗi nhân / 512 KiB
On-chip SRAM	256 MiB (theo bài)
DRAM	1 GiB DDR3
I/O	GbE, micro-SD, HDMI

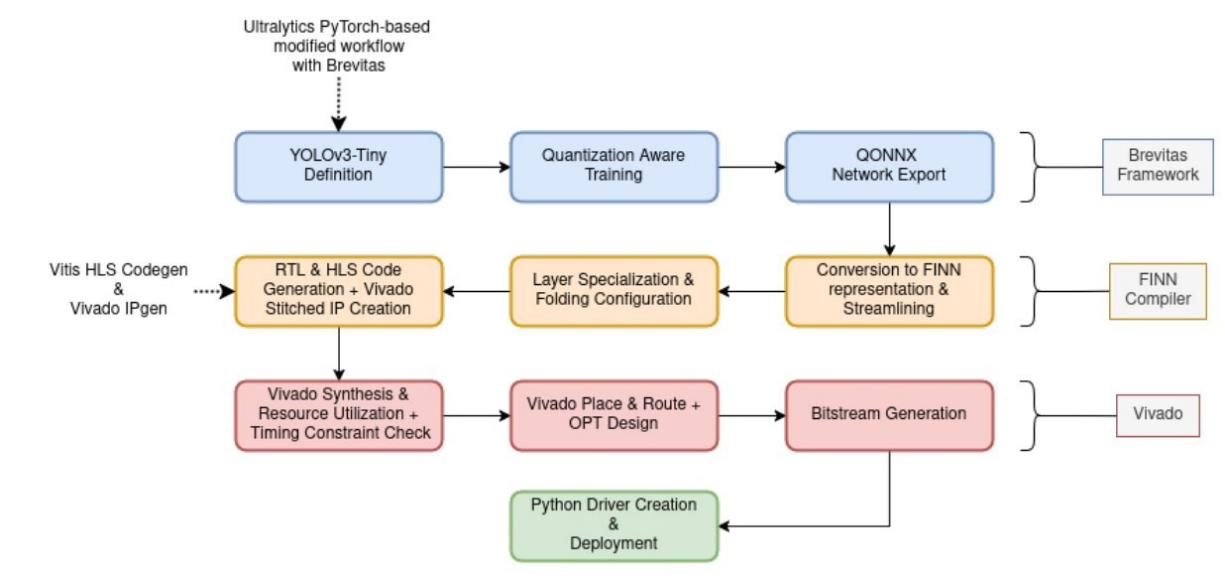
Logic lập trình (PL)	Giá trị
LUT	53 200
Flip-flop	106 400
BRAM	140 × 36 Kb (trong bảng dùng 140 BRAM_18K)
DSP48E1	220 (MAC 48 bit)
Giao diện AXI	4× HP 64 bit tới DRAM; 4× AXI 32 bit điều khiển

Họ Zynq-7000 dựa trên Artix-7, công suất vài watt, giá chip khoảng 100 €.

Lưu ý: bài ghi 140×36 Kb trong mục 3 nhưng bảng 7 dùng 140 BRAM_18K, không nhất quán.

QUY TRÌNH

Quy trình thiết kế đầu-cuối (Hình 1)



Brevitas (QAT, xuất QONNX) → FINN (biến đổi, đặc tả lớp, folding, sinh IP) → Vivado (tổng hợp, P&R, bitstream) → driver Python.

MÔ HÌNH

Điều chỉnh YOLOv3-Tiny cho FINN

Thay đổi	Lý do	Hệ quả
Bỏ upsample và concat	FINN chưa hỗ trợ topo rẽ nhánh	Mất nhánh phát hiện thứ hai (đa tỉ lệ)
Số kênh mọi tích chập /5	Trọng số phải vừa BRAM trên chip	Giảm năng lực biểu diễn
Tách lớp detect khỏi đồ thị	Chỉ là phép đổi hệ tọa độ hộp, hợp float trên PS	Phần hậu xử lý nằm trên ARM
Đầu vào 416×416, một lớp (car)	Bám LPYOLO và bài toán ô tô nhìn từ trên	Không so được mAP với công trình khác

Hệ quả được tác giả thừa nhận: recall giảm một nửa vì mất một nhánh phát hiện.

KIẾN TRÚC

Mạng sau thu gọn (Bảng 1)

Lớp	Vào	Ra	K	S	Act
Conv	416×416×3	416×416×8	3	1	ReLU
MaxPool	416×416×8	208×208×8	2	2	-
Conv	208×208×8	208×208×8	3	1	ReLU
MaxPool	208×208×8	104×104×8	2	2	-
Conv	104×104×8	104×104×16	3	1	ReLU
MaxPool	104×104×16	52×52×16	2	2	-
Conv	52×52×16	52×52×32	3	1	ReLU
MaxPool	52×52×32	26×26×32	2	2	-

Lớp	Vào	Ra	K	S	Act
Conv	26×26×32	26×26×56	3	1	ReLU
MaxPool	26×26×56	13×13×56	2	2	-
Conv	13×13×56	13×13×104	3	1	ReLU
MaxPool	13×13×104	13×13×104	2	2*	-
Conv	13×13×104	13×13×208	3	1	ReLU
Conv	13×13×208	13×13×56	1	1	ReLU
Conv	13×13×56	13×13×104	3	1	ReLU
Conv	13×13×104	13×13×18	3	1	HardTanh

** Bảng 1 của bài ghi stride 2 ở MaxPool 13×13 dù kích thước vào/ra không đổi (nhiều khả năng là stride 1; ghi chú của người trình bày). K: kernel, S: stride, Act: kích hoạt Quant (QuantReLU/QuantHardTanh); lớp cuối 18 kênh = 3 anchor × (4 hộp + 1 objectness + 1 lớp) là diễn giải của người trình bày.*

Quantization-Aware Training với Brevitas

Cách làm

- Lấy YOLOv5 Ultralytics làm khung huấn luyện
- Thay Conv/Linear bằng lớp Brevitas
- Thêm module quant_common.py

Chọn số bit

- Trọng số lớp đầu và cuối: INT8 (giữ thông tin ở đầu vào, đầu ra chi tiết)
- Các lớp còn lại: INT4 cho trọng số và kích hoạt
- Kích hoạt: QuantReLU, QuantHardTanh ở lớp cuối

Thành phần	Độ rộng bit
Trọng số conv đầu và cuối	8
Kích hoạt lớp đầu	4
Trọng số và kích hoạt các lớp giữa	4 và 4

Mô phỏng sai số lượng tử ngay trong huấn luyện giúp mạng học trọng số bền với độ chính xác thấp.

VisDrone, chỉ lớp car

Tập dữ liệu

- 10 209 ảnh, 2,6 triệu hộp, 10 lớp
- Dùng: task 1 và chỉ lớp car
- Chú thích đổi sang định dạng YOLO XYWH

Môi trường

- Kaggle Notebook, Tesla P100 16 GB
- torch 1.13.1, torchvision 0.14.1
- brevitas 0.9.0, numpy 1.26.3, opencv 4.10

Tham số	Giá trị
Epoch	300
Batch size	64
Kích thước ảnh	416×416
Optimizer	SGD
Patience	30

Hyperparameter chuẩn của YOLOv5 'from scratch', không tinh chỉnh.

XUẤT MÔ HÌNH

ONNX cho FINN và baseline cho Coral

Nhánh FPGA

- export.py --include finn_onnx sinh QONNX
- Lớp detect bị tách ra khỏi đồ thị
- Mạng chứa nút Quant đặt trước Conv/MaxPool

Nhánh Coral (đối chứng)

- Huấn luyện lại ở FP32
- Xuất TFLite với PTQ int8
- Biên dịch bằng edgetpu compiler, dùng libedgetpu bản cộng đồng

Điều cần lưu ý

Hai nhánh không cùng một mô hình: FPGA dùng QAT INT4 với mạng đã thu gọn, Coral dùng PTQ int8. Bài không báo cáo mAP của mô hình Coral nên so sánh tốc độ không đi kèm so sánh độ chính xác.

BIẾN ĐỔI FINN (1)

Từ QONNX đến mạng đã streamline

ConvertQONNXtoFINN

Gộp nút Quant vào Conv/MaxPool; ReLU và activation thành MultiThreshold

Tidy-up

Ghi shape, kiểu dữ liệu, đặt tên duy nhất cho từng nút

Chuẩn hóa đầu vào

Chèn nút Div trước conv đầu: INT8 về [0, 1]

Streamline

Gom toán tử float, chuyển thành MultiThreshold

Lowering

Conv thành Im2Col + MatMul

Ánh xạ phần cứng và dataflow partition

Ánh xạ

- MatMul thành MVU (Matrix-Vector Unit)
- Im2Col thành SWG/CIG (Sliding Window Generator)
- MaxPool, Threshold thành khối streaming

Phần chạy trên PS

- Hai Transpose (đổi bố cục dữ liệu YOLO và FINN)
- Phép Mul tỉ lệ để khử lượng tử đầu ra
- Hậu xử lý: detect, NMS, vẽ hộp

CreateDataflowPartition

FINN hiện chỉ ánh xạ được một partition lên PL, vì vậy mọi thứ không tổng hợp được phải ở rìa đồ thị. Điều này ràng buộc ngược lại thiết kế mạng: không có nhánh rẽ giữa mạng, và các nút cuối chỉ là phép tỉ lệ đơn giản.

RTL hay HLS cho MVU và SWG

Thành phần	RTL	HLS
MVU	Dùng DSP48 với DSP packing; chưa hỗ trợ kiểu LUT	Hỗ trợ cả DSP và LUT nhưng không đóng gói, tốn tài nguyên hơn
SWG (CIG)	Parallel window: xử lý đồng thời cửa sổ mọi kênh, giảm chu kỳ, đòi hỏi SIMD unfold hoàn toàn	Không có parallel window
Khi nào dùng	Mặc định, ưu tiên	Khi hết DSP: dùng LUT dư để unfold thêm MVU

Tác giả viết một script Python 3.10 để chuyển qua lại kiểu đặc tả từng lớp khi dò cấu hình tối ưu.

DSP PACKING

Số MAC trên mỗi DSP (Bảng 4)

Loại DSP	Kích hoạt (U)INT4 × trọng số INT4	Kích hoạt (4,8] bit × trọng số (4,8] bit
HLS DSP	1 MAC/DSP	1 MAC/DSP
RTL DSP48E1 (Zynq-7000)	**4 MAC/DSP**	**2 MAC/DSP**
RTL DSP48E2	4 MAC/DSP	2 MAC/DSP
RTL DSP58	3 MAC/DSP	3 MAC/DSP

Ý nghĩa

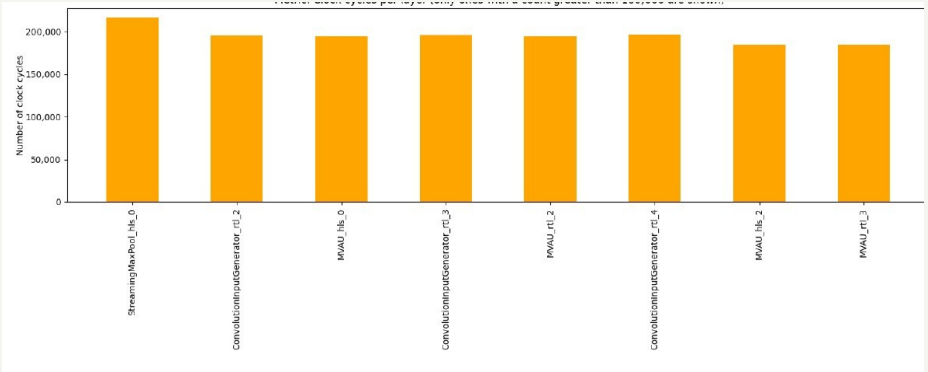
Hạ trọng số và kích hoạt xuống 4 bit không chỉ giảm bộ nhớ: nó cho phép nhồi bốn phép nhân vào một DSP48E1. Với chỉ 220 DSP, đây là đòn bẩy chính cho thông lượng. Phần LUT còn lại được dùng cho MVU HLS để unfold thêm.

FOLDING

Cân bằng chu kỳ giữa các lớp

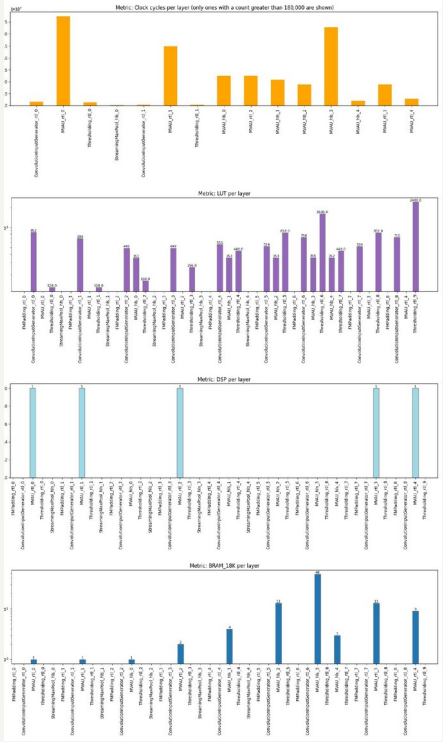
Thủ tục lặp

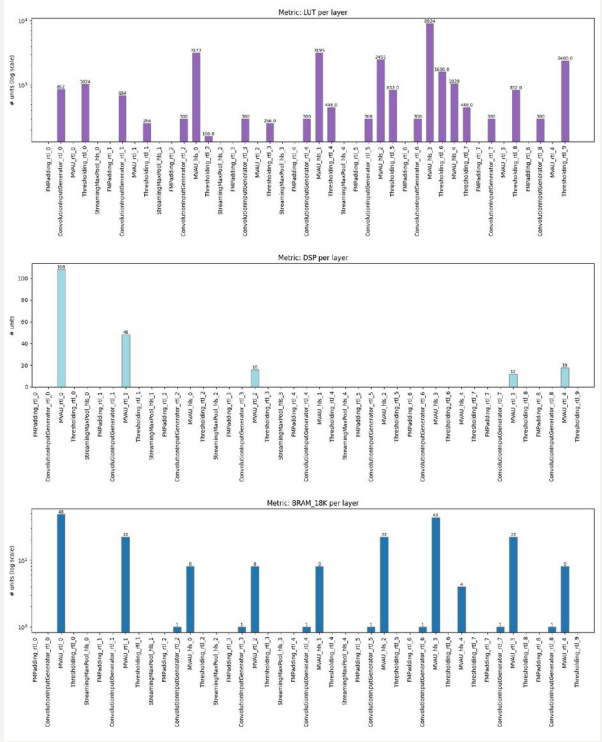
- Bắt đầu: mọi SIMD = PE = 1 (fold hoàn toàn)
- Áp cấu hình JSON bằng ApplyConfig
- Mỗi vòng: unfold lớp chậm nhất
- Cân bằng chu kỳ trên mỗi lớp trong giới hạn tài nguyên
- Lớp nhỏ (MaxPool, SWG, MultiThreshold cuối mạng) giữ fold hoàn toàn
- Dừng khi lớp chậm nhất đã unfold hết cỡ



Hình 10: số chu kỳ mỗi lớp sau tối ưu. Lớp dài nhất là StreamingMaxPool_hls_0 (~216 000 chu kỳ theo dự đoán); các lớp lớn được đưa về mức xấp xỉ nó.

Mặc định và tối ưu (Hình 8, 11)





Cách đọc

- Bốn biểu đồ cột: chu kỳ, LUT, DSP, BRAM_18K theo lớp
- Hình 8 (trái): cấu hình mặc định
- Hình 11 (giữa): cấu hình tối ưu
- MVU lớn dùng HLS-LUT để giữ DSP dưới 220
- Estimate của FINN: LUT chỉ là ước tính vì Vivado tối ưu lại

Độ sâu FIFO và lỗi StreamingMaxPool (HLS)

Bài toán FIFO

- Các lớp có số chu kỳ khác nhau, cần đệm giữa chúng
- Quá ngắn: stall nhiều; quá dài: phí BRAM
- Tìm bằng mô phỏng RTL (pyverilator, Verilator)

Con số

- 570 000 chu kỳ thực (Vitis HLS)
- 216 320 chu kỳ dự đoán
- Chênh 2,6×; thông lượng giảm 3,43×

Nguyên nhân và hệ quả

Theo tác giả, Vitis không tối ưu unroll vòng lặp trên bản đồ đặc trưng lớn, làm thời gian chạy tăng vọt. Nút thắt này một mình làm thông lượng thực tế giảm 3,43 lần. Viết lại bằng RTL sẽ khắc phục nhưng nằm ngoài phạm vi bài. Bản FINN dùng ở đây chỉ có MaxPool HLS cho dạng này.

Trọng số trong BRAM, quy trình tổng hợp

Trọng số trong BRAM

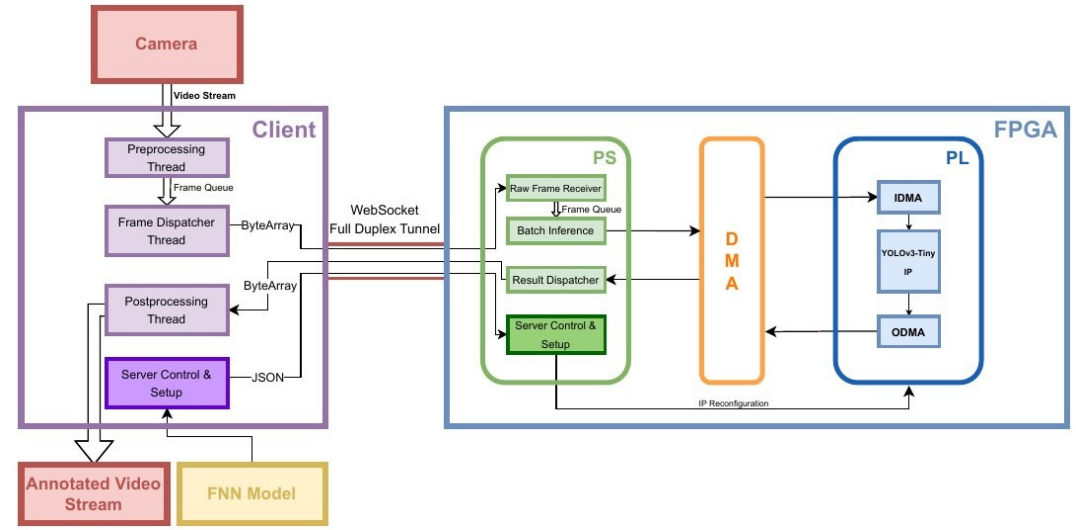
- Truy cập nhanh, không qua DMA/DRAM ngoài
- Tránh thêm công suất truy cập DRAM
- Chọn lúc folding: nội bộ hay ngoại bộ

Số liệu (tính từ bảng 6)

- 4 bit: 333 632 trọng số; 8 bit: 17 064
- Tổng ≈ 1,47 Mbit ≈ 184 KB
- BRAM dùng: 138 × 18 Kb ≈ 2,48 Mbit

Bước	Transformation	Kết quả
1	PrepareIP	Xuất mã RTL/HLS vào thư mục build
2	HLSSynthIP	Vitis HLS tổng hợp sang RTL (lâu)
3	CreateStitchedIP	Ghép IP; thêm idma và odma tới DRAM
4	ZynqBuild	Vivado tổng hợp, P&R (lâu nhất)

Kiến trúc client–server (Hình 14)



Luồng ở client

- Capture: OpenCV, tiền xử lý, hàng đợi
- Frame Dispatcher: gói thông điệp nhị phân gửi WebSocket
- Post-processing: ghép kết quả, vẽ hộp
- Display: hiển thị video

Giao thức, định dạng .fnn và xử lý lô

Trường header (16 byte)	Kích thước
frame_id	4 byte (int)
width	4 byte (int)
height	4 byte (int)
channels	2 byte (short)
dtype_code	2 byte (short)

Xử lý lô

Server gom tối đa 100 khung rồi chạy trong một worker, tận dụng pipeline. Nạp mô hình: giải mã base64, kiểm tra tệp, nạp bitstream bằng PYNQ, khởi tạo hàng đợi.

Lệnh JSON

- load_model: nạp mô hình .fnn mới lên board
- status: trạng thái nạp, kích thước hàng đợi, số client
- Kiểm tra toàn vẹn bằng MD5

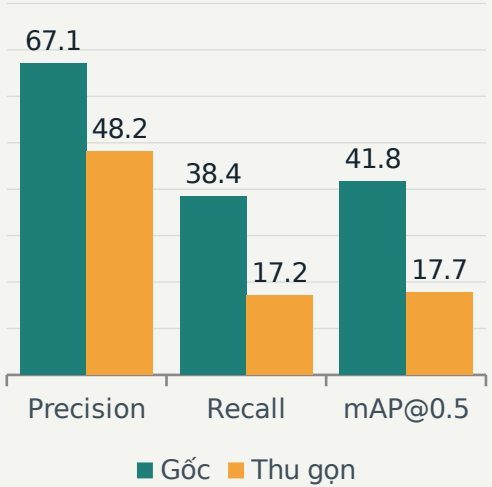
Tệp .fnn (zip)

- model.bit: bitstream
- model.hwh: mô tả phần cứng
- model.pkl: shape và metadata
- model_info.json: tần số tối đa, tên, tập huấn luyện

KẾT QUẢ: ĐỘ CHÍNH XÁC

Kết quả huấn luyện (Bảng 5)

Chỉ số	YOLOv3-Tiny gốc	Bản thu gọn
Precision	67,06%	48,16%
Recall	38,44%	17,16%
mAP@0.5	41,76%	17,68%
Box loss	0,1220	0,1531
Object loss	0,1653	0,3049
Classification loss	0	0

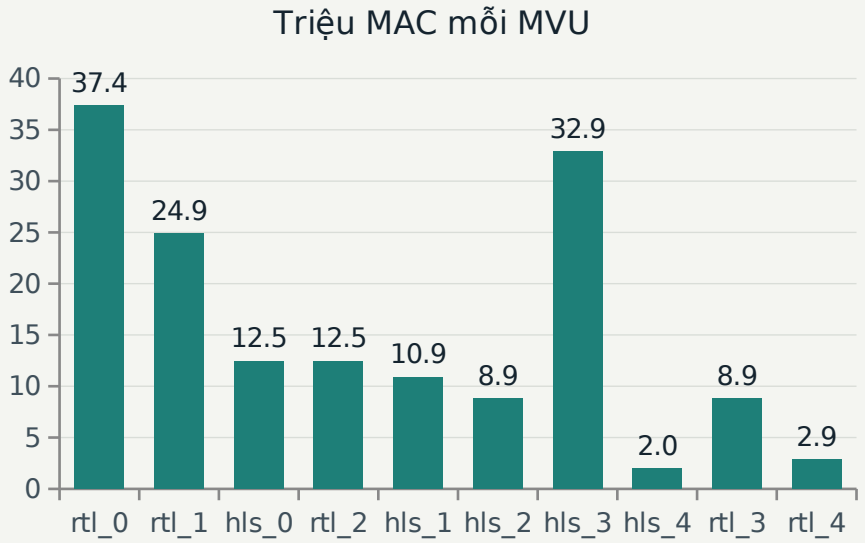


Lớp phân loại bằng 0 vì chỉ có một lớp (car). Precision và recall ghi theo epoch cuối trên tập kiểm định.

KẾT QUẢ: TÍNH TOÁN

MAC và trọng số theo từng MVU (Bảng 6)

Lớp	MAC	Trọng số	Kiểu
MVAU_rtl_0	37 380 096	216	8b-8b
MVAU_rtl_1	24 920 064	576	4b-4b
MVAU_hls_0	12 460 032	1 152	4b-4b
MVAU_rtl_2	12 460 032	4 608	4b-4b
MVAU_hls_1	10 902 528	16 128	4b-4b
MVAU_hls_2	8 858 304	52 416	4b-4b
MVAU_hls_3	32 902 272	194 688	4b-4b
MVAU_hls_4	1 968 512	11 648	4b-4b
MVAU_rtl_3	8 858 304	52 416	4b-4b
MVAU_rtl_4	2 847 312	16 848	4b-8b

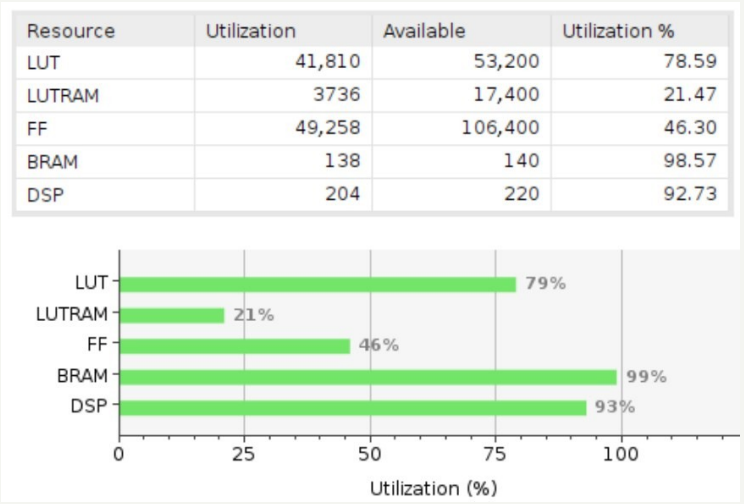


Lớp đầu (conv 3×3 trên ảnh 416×416, trọng số 8 bit) chiếm nhiều MAC nhất nhưng có rất ít trọng số; lớp MVAU_hls_3 có nhiều trọng số nhất (194 688).

Tài nguyên PL: ước tính và thực tế (Bảng 7)

Tài nguyên	Ước tính	Thực tế	Sẵn có	%
BRAM_18K	200	138	140	98,57
LUT	26 694	41 605	53 200	78,20
DSP	202	204	220	92,73
URAM	0	-	-	-

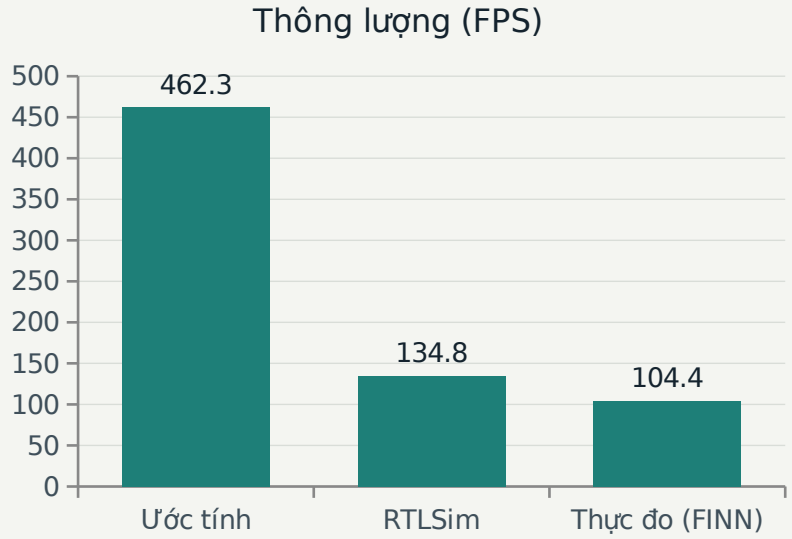
Chênh lệch ước tính-thực tế: BRAM thấp hơn dự kiến (200 → 138) vì một phần bộ nhớ bị Vivado chuyển sang LUT-RAM (3 736 LUTRAM); LUT cao hơn (26 694 → 41 605) vì HLS và vì bù cho BRAM.



Hình 16: Vivado, LUT 79%, LUTRAM 21%, FF 46%, BRAM 99%, DSP 93%.

KẾT QUẢ: HIỆU NĂNG

Ước tính, RTLSim và thực đo (Bảng 8)



Bài ghi độ trễ ước tính 45,87 ms (tổng chu kỳ) nhưng đo thực 11,56 ms ở batch 1: tổng chu kỳ các lớp không phải độ trễ của pipeline.

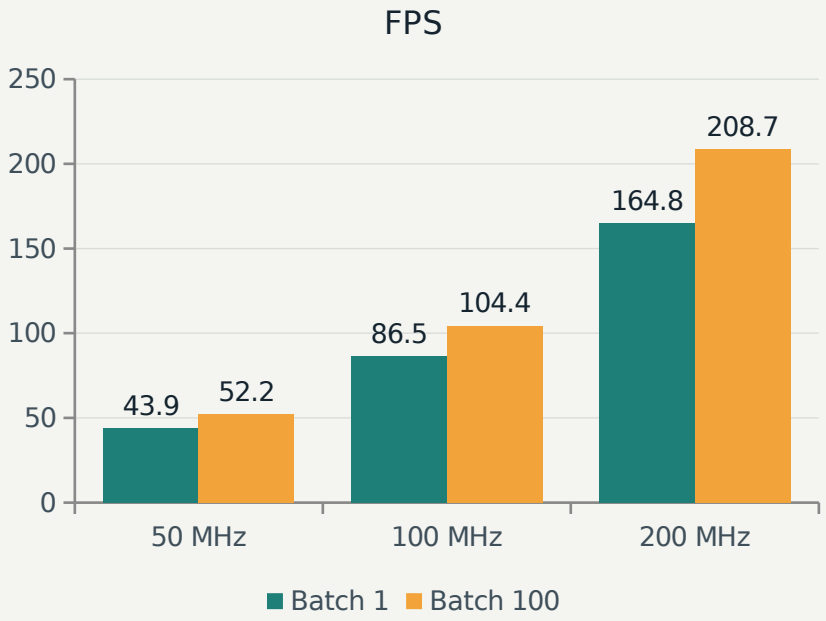
Chỉ số	Ước tính	RTLSim	Thực đo
Max chu kỳ (nút)	216 320	741 670	-
Độ trễ (ms)	45,87	7,41	11,56
FPS	462,28	134,83	104,39

Vì sao thực đo thấp hơn

- Ước tính bỏ qua giao dịch bộ nhớ, ranh giới pipeline, stall
- RTLSim không thấy overhead driver và độ trễ phía PS
- MaxPool HLS: 741 670 chu kỳ thay vì 216 320 (3,43× chậm)

KẾT QUẢ: HIỆU NĂNG

Thông lượng theo tần số và lô (Bảng 9)



MHz	Lô	ms	FPS	DRAM Mb/s
50	1	22,77	43,91	22,8
50	100	1915,5	52,21	27,1
100	1	11,56	86,47	44,9
100	100	958,0	104,39	54,2
200	1	6,07	164,79	85,6
200	**100**	**479,2**	**208,69**	**108,3**

Batch 100 làm đầy pipeline; với batch 1 phải chờ xả pipeline. Tuyến tính theo clock, nên băng thông DRAM vẫn còn dư (≈108 Mb/s ở 200 MHz).

KẾT QUẢ: OVERHEAD

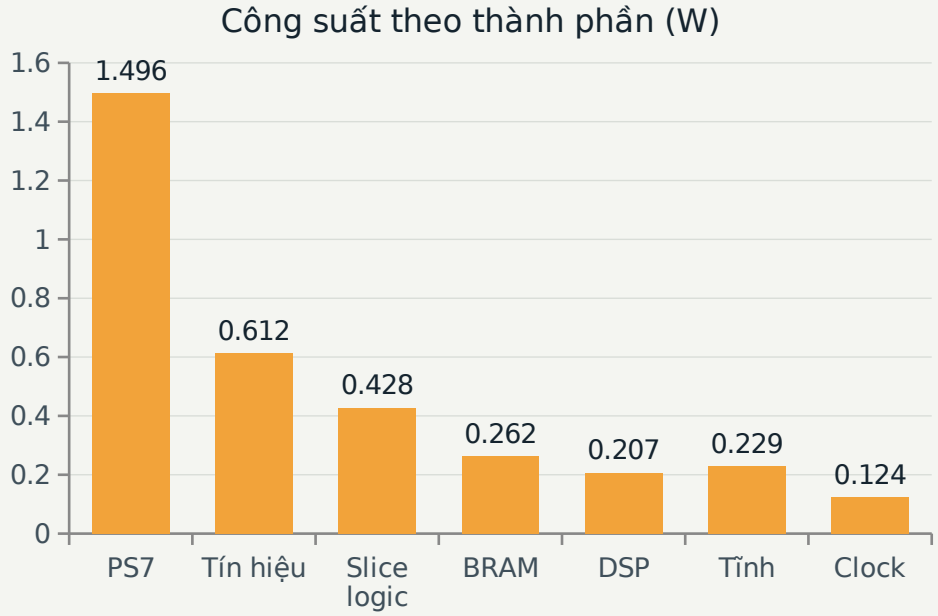
Thời gian từng giai đoạn (Bảng 10)

Giai đoạn (ms)	50 MHz	100 MHz	200 MHz
Tiền xử lý	32,9 ± 1,0	34,0 ± 1,2	32,2 ± 1,5
Driver exec	28,9 ± 0,7	17,9 ± 0,8	12,1 ± 0,3
Rescale	0,51 ± 0,21	0,50 ± 0,18	0,48 ± 0,02
Detect	4,99 ± 1,14	4,67 ± 0,24	4,75 ± 4,50
NMS	10,4 ± 2,6	9,34 ± 1,05	9,23 ± 2,38
Box process	11,4 ± 2,8	11,2 ± 3,2	10,7 ± 2,95
Tổng hậu xử lý	**28,0 ± 5,0**	**26,4 ± 3,6**	**25,8 ± 8,8**

Nhận xét

- Driver exec = suy luận + copy + pack/unpack
- Phần cố định không co theo clock → FPS thực giảm 21% (50 MHz), 35% (100), 50% (200)
- Giai đoạn PS-bound chiếm phần lớn

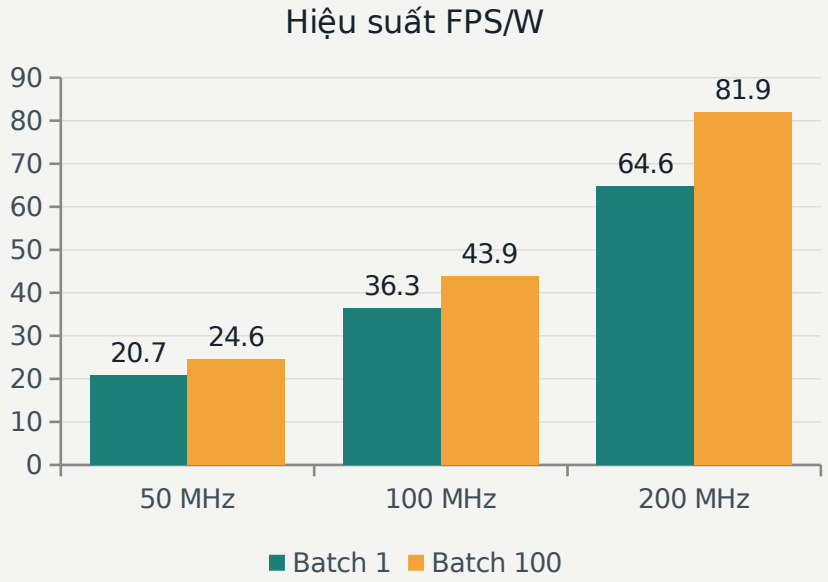
Công suất ước tính Vivado (Bảng 11-12)



Khối (Bảng 11)	W
Tổng top_wrapper	3,129
StreamingDataflowPartition	1,554
zynq_ps	1,497
smartconnect	0,042
idma + odma	0,031

Phần mạng (1,554 W) và PS (1,497 W) gần ngang nhau trong ước tính; trong thực đo, PS chiếm áp đảo.

Công suất đo và FPS/W (Bảng 13, 14)



Cấu hình	Công suất
Rảnh (chưa nạp bitstream)	1,90 W
50 MHz	2,12 W
100 MHz	2,38 W
200 MHz	2,55 W

PL thật sự chỉ thêm 0,22–0,65 W. Phần 1,9 W là PS + DRAM, chưa giảm xung khi rảnh. Driver bare-metal sơ bộ tiết kiệm trên 20% công suất so với Linux.

Chú ý: FPS/W dùng công suất cả board, không tách PS.

KẾT QUẢ: HỆ THỐNG

Thông lượng và độ trễ đầu-cuối qua mạng

12 ms

suy luận mỗi khung, 200 MHz, driver FINN

13 ms

độ trễ luồng tối đa, Ethernet có dây

130 ms

độ trễ luồng tối đa qua WiFi

30 FPS

camera trực tiếp, macOS client

Vì sao 30 chứ không 208 FPS

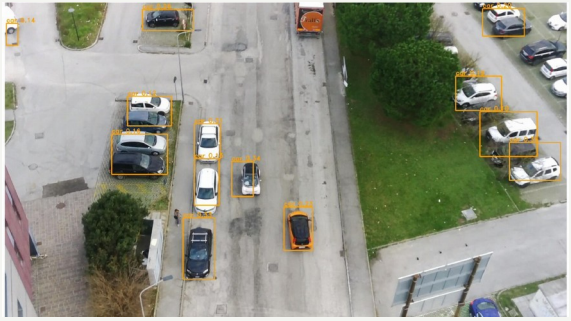
- Sao chép buffer thô sang numpy cho mỗi khung
- Chuyển đổi định dạng trong Python
- Phụ thuộc mạng và client

So với LPYOLO

- LPYOLO cùng cấu hình mạng và board: 18 FPS
- Bài này: 30 FPS
- Tác giả kỳ vọng bản C của driver sẽ giảm overhead mạnh

KẾT QUẢ: THỰC ĐỊA

Thử nghiệm với video drone thật (Hình 19)







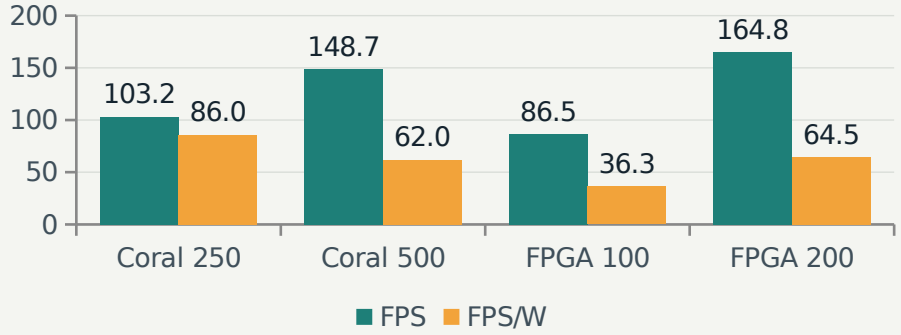
Dù recall trên tập kiểm định chỉ 17%, hầu hết ô tô trong cảnh thực địa được phát hiện và định vị đúng. Lý giải của tác giả: khi bay ở độ cao tiêu chuẩn, kích thước vật thể gần như cố định, nên một nhánh ở một tỉ lệ là đủ.

Hạn chế: không có đánh giá định lượng (precision/recall) trên video thực địa.

SO SÁNH 1

FPGA so với Coral Edge TPU (Bảng 15)

Batch 1	Coral 250	Coral 500	FPGA 50	FPGA 100	FPGA 200
Độ trễ (ms)	9,69	6,73	22,77	11,56	6,07
FPS	103,2	148,7	43,91	86,47	164,8
Công suất (W)	1,2†	2,4†	2,12	2,38	2,55
FPS/W	86,0	62,0	20,7	36,3	64,5

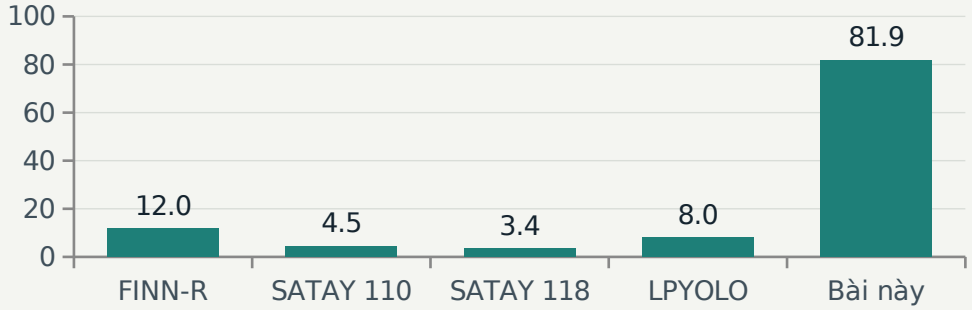


† Ước tính từ datasheet, không đo. Coral dùng mô hình riêng (FP32 → PTQ int8), không báo cáo mAP. FPGA ở batch 100 đạt 208,7 FPS, 81,9 FPS/W, nhưng bài chọn batch 1 cho so sánh công bằng.

SO SÁNH 2

So với các FPGA YOLOv3-Tiny khác (Bảng 16)

Công trình	Board	Bit	Clock	LUT	DSP	FPS	W	FPS/W
FINN-R	PYNQ-Z1	1W3A	100	46,5K	-	29,9	2,5	11,96
SATAY	VCU110	8W16A	220	127K	1780	69,9	15,4	4,54
SATAY	VCU118	8W16A	255	431K	6687	147,1	42,9	3,43
LPYOLO	Z7020	4W4A	100	39,4K	203	19,1	2,4	7,97
Bài này	**Z7020**	**4W4A**	**200**	**37,9K**	**204**	**208,8**	**2,55**	**81,85**



VCU118 giá trên 15 000 € so với ~150 € cho Z-Turn. SATAY dùng 8 bit trọng số và 16 bit kích hoạt, mạng nặng hơn.

PHÂN TÍCH

Số liệu tự tính từ các bảng của bài

64 GOPS

153,6 triệu MAC × 2 × 208,7 FPS

25 GOPS/W

64 GOPS / 2,55 W (cả board)

≈98 GOPS/W

64 GOPS / 0,65 W (chỉ phần PL thêm vào)

Tỉ số FPS/W theo Bảng 16

- vs FINN-R: 81,85 / 11,96 = 6,8×
- vs LPYOLO: / 7,97 = 10,3×
- vs SATAY VCU110: / 4,54 = 18,0×
- vs SATAY VCU118: / 3,43 = 23,9×

Bộ nhớ trọng số

- 333 632 × 4 bit + 17 064 × 8 bit ≈ 1,47 Mbit
- 138 BRAM_18K ≈ 2,48 Mbit dung lượng
- BRAM hết 98,6% chủ yếu vì độ rộng/độ sâu truy cập song song, không phải dung lượng thô

PHẢN BIỆN

Điểm mạnh và các điểm cần thận trọng

Điểm mạnh

- Quy trình đầu-cuối, mã nguồn công khai
- Đo công suất thật và có phân tích chênh lệch với Vivado
- Thừa nhận minh bạch nút thắt MaxPool HLS và driver Python
- Bóc tách overhead từng giai đoạn (Bảng 10)
- Nhắm chip rẻ (~100–150 €), có ý nghĩa thực tiễn

Cần thận trọng

- mAP@0.5 17,7% chỉ cho lớp car, mạng bị cắt nhiều; không đánh giá định lượng trên video thực địa
- Công suất Coral lấy từ datasheet; mô hình Coral khác, không báo cáo mAP
- Mức cải thiện FPS/W không nhất quán (3–7×, 7–24×, 7,3×); Bảng 16 cho 6,8–24×
- Bảng 16 ghi độ trễ 4,79 ms; Bảng 15 ghi 6,07 ms (batch 1)
- LUT: 37,9K (Bảng 16) vs 41,6K (Bảng 7)
- FPS/W dùng cả board: PS chiếm 1,9 W

KẾT LUẬN

FPGA giá rẻ chạy được YOLOv3-Tiny

Đóng góp

- Quy trình đầu-cuối trên Zynq-7020
- 208 FPS, 2,55 W, tương đương Coral Edge TPU
- FPS/W cao hơn các FPGA khác, ~10× FPS so với LPYOLO

Bài học kỹ thuật

- INT4 + DSP packing: 4 MAC/DSP
- Cân bằng chu kỳ giữa lớp quyết định thông lượng
- Một lớp HLS kém có thể kéo cả pipeline

Hướng tiếp theo (theo tác giả)

- Driver C bare-metal: tiết kiệm trên 20% công suất, DMA song song với suy luận, đưa overhead về gần 0
- Mô hình lớn hơn, tối ưu bộ nhớ
- Mở rộng sang UAV và IoT; viết lại MaxPool bằng RTL

Hỏi và đáp

Vì sao chọn 4 bit?

Cho 4 MAC/DSP48E1 và vừa BRAM; lớp đầu/cuối 8 bit để giữ thông tin.

Tại sao chỉ 30 FPS đầu-cuối?

Driver Python, copy buffer, mạng; phần cứng đạt 208 FPS.

Mô hình có dùng được đa lớp?

Bài chỉ huấn luyện lớp car; mạng thu gọn một nhánh, recall giảm một nửa.

Công suất có thấp thật?

Cả board 2,55 W, trong đó PS+DDR ~1,9 W; PL chỉ thêm 0,22–0,65 W.

Có tái lập được không?

Mã và notebook công khai (repo [62]); phiên bản FINN 0.10, Brevitas 0.9.0.